

Univerzitet u Sarajevu
Elektrotehnički fakultet

Odsjek za automatiku i elektroniku

Predmet: **Strukture i režimi rada elektroenergetskih sistema**

Konvertor trofazne estimacije stanja u pravougaonim koordinatama

Projektni izvještaj

Student: Tarik Čatić

Broj indeksa: 19786

Profesor: Izudin Džafić

Sarajevo, 2026.

Sadržaj

1	Uvod	2
2	Teorijske osnove	2
2.1	Estimacija stanja elektroenergetskog sistema	2
2.2	Metoda težinskih najmanjih kvadrata (WLS)	2
2.3	Polarne i pravougaone koordinate	3
2.4	Trofazni sistemi i nebalansiranost	3
3	Postavka zadatka i ulazni podaci	3
3.1	Zahtjevi	3
3.2	Format ulaznog fajla	3
4	Arhitektura i implementacija	4
4.1	Struktura plugina	4
4.2	Parsiranje ulaznog fajla	4
4.3	Konverzija u radnoj niti	5
4.4	Višenitnost i prikaz napretka	5
4.5	Generisanje izlaznog modela	6
4.6	Korisnički interfejs	6
5	WLS estimacija u dTwinu	6
5.1	Dodavanje mjernog šuma	6
5.2	Statistika greške	7
6	Rezultati i analiza	7
6.1	Test slučajevi	7
6.2	Provjera tačnosti konverzije	7
6.3	Nebalansirani slučajevi	9
6.4	Istorija rješavanja	11
6.5	Analiza greške estimacije	12
7	Zaključak	13

1 Uvod

Estimacija stanja (*engl. state estimation*) jedan je od temeljnih zadataka u nadzoru i vođenju elektroenergetskih sistema (EES). Njena je uloga da iz skupa redundantnih i šumom opterećenih mjerenja odredi najvjerojatnije stvarno stanje mreže – prije svega napone u čvorovima – koje potom služi kao ulaz za analize sigurnosti, proračun tokova snaga i optimizaciju pogona.

Cilj ovog projekta jeste izrada **konvertora** koji ulazne trofazne napone, zadate u polarnom obliku (magnituda i ugao po fazi), pretvara u **pravougaone (Dekartove) koordinate** i na osnovu njih generiše kompletan model težinske metode najmanjih kvadrata (WLS) pogodan za rješavanje u alatu **Digital Twin (dTwin)**. Konverzija se izvršava u zasebnoj radnoj niti, dok se u drugoj niti u realnom vremenu prikazuje napredak, uz korisnički interfejs realizovan kao DLL dodatak (*plugin*) za dTwin.

Rad je organizovan na sljedeći način. U poglavlju 2 date su teorijske osnove estimacije stanja, WLS metode te odnosa polarnih i pravougaonih koordinata. Poglavlje 3 opisuje postavku zadatka i format ulaznih podataka. Poglavlje 4 detaljno prikazuje arhitekturu i implementaciju plugina. Poglavlje 5 opisuje strukturu WLS modela i dodavanje mjernog suma, a poglavlje 6 donosi rezultate i analizu. Zaključci su dati u poglavlju 7.

2 Teorijske osnove

2.1 Estimacija stanja elektroenergetskog sistema

U realnoj mreži broj raspoloživih mjerenja (m) veći je od broja nepoznatih veličina stanja (n), tj. sistem je *predodređen* ($m > n$). Mjerenja su usto opterećena greškom mjernih uređaja. Estimacija stanja stoga nije obično rješavanje sistema jednačina, već traženje onog vektora stanja koji *najbolje* objašnjava sva mjerenja u statističkom smislu.

Veza mjerenja $\mathbf{z}$ i stanja $\mathbf{x}$ data je mjernim modelom

$$\mathbf{z} = \mathbf{h}(\mathbf{x}) + \mathbf{e}, \quad (1)$$

gdje je $\mathbf{h}(\cdot)$ vektor mjernih funkcija, a $\mathbf{e}$ vektor grešaka mjerenja, najčešće modelovan kao Gausov šum sa nultom srednjom vrijednošću.

2.2 Metoda težinskih najmanjih kvadrata (WLS)

Najčešće korišten estimator jeste metoda težinskih najmanjih kvadrata (WLS). Ona minimizira težinsku sumu kvadrata odstupanja (reziduala) mjerenja od modelom predviđenih vrijednosti:

$$J(\mathbf{x}) = \sum_{i=1}^m w_i (z_i - h_i(\mathbf{x}))^2, \quad (2)$$

gdje je w_i težina mjerenja i , obrnuto srazmjerna varijansi tog mjerenja, pa pouzdanija mjerenja imaju veći uticaj na rješenje. Minimum funkcije (2) nalazi se iterativno (Gauss–

Newton), rješavanjem tzv. normalnih jednačina

$$\mathbf{G} \Delta \mathbf{x} = \mathbf{H}^T \mathbf{W} (\mathbf{z} - \mathbf{h}(\mathbf{x})), \quad \mathbf{G} = \mathbf{H}^T \mathbf{W} \mathbf{H}, \quad (3)$$

gdje je $\mathbf{H}$ Jakobijeva matrica mjerenja, $\mathbf{W}$ dijagonalna matrica težina, a $\mathbf{G}$ tzv. *gain* matrica. Zbog rijetke (*sparse*) strukture mreže, matrica $\mathbf{G}$ je rijetka i faktorizuje se rijetkim metodama – što je posao internog solvera alata dTwin.

2.3 Polarne i pravougaone koordinate

Napon čvora se može predstaviti kompleksnim brojem. U *polarnom* obliku zadaje se magnitudom V_m i uglom V_a , a u *pravougaonom* realnim i imaginarnim dijelom:

$$\underline{V} = V_m \angle V_a = e + j f, \quad e = V_m \cos V_a, \quad f = V_m \sin V_a. \quad (4)$$

Veličine e i f (u ovom radu označene i kao V_r i V_i) predstavljaju promjenljive stanja u pravougaonoj formulaciji. Prednost pravougaonih koordinata jeste da mrežne jednačine postaju *polinomske* (bez trigonometrijskih funkcija), što čini Jakobijan jednostavnijim i numerički stabilnijim. Upravo je pretvorba (4) jezgro ovog konvertora.

2.4 Trofazni sistemi i nebalansiranost

Kod trofaznih sistema svaka sabirnica ima po tri napona – za faze A, B i C. U idealno balansiranom sistemu magnitude su jednake, a uglovi razmaknuti za 120° (0° , -120° , $+120^\circ$). U stvarnim mrežama, zbog nesimetričnih opterećenja i vodova, javlja se *nebalansiranost* – male razlike magnituda i odstupanja uglova od nominalnih. Konvertor obrađuje sve tri faze nezavisno, čime je omogućena i obrada nebalansiranih slučajeva.

3 Postavka zadatka i ulazni podaci

3.1 Zahtjevi

Zadatak obuhvata tri osnovna zahtjeva:

1. konverziju napona $V, \theta \rightarrow V_r, V_i$ po fazi, realizovanu u C++ u zasebnoj radnoj niti;
2. prikaz napretka konverzije u realnom vremenu, u drugoj niti, na *thread-safe* način;
3. grafički korisnički interfejs i isporuku kao DLL plugin za dTwin.

3.2 Format ulaznog fajla

Ulaz je MATPOWER (.m) datoteka koja sadrži trofazni blok napona. Parser prihvata **dva rasporeda** i interno ih svodi na isti oblik.

Prošireni format **bus3ph** (7 kolona), korišten u test-fajlovima ovog projekta:

```
mpc.bus3ph = [
    busId  Vm_A  Va_A  Vm_B  Va_B  Vm_C  Va_C;
    ...
];
```

Standardni MATPOWER format `bus3p` (9 kolona), kakav se pojavljuje u stvarnim slučajevima (npr. `t_case3p_a.m`):

```
mpc.bus3p = [
    busid  type  basekV  Vm1  Vm2  Vm3  Va1  Va2  Va3;
    ...
];
```

Ovdje su magnitude grupisane prije uglova; faze 1/2/3 odgovaraju fazama A/B/C. Sve ostale MATPOWER blokove (`mpc.bus`, `mpc.branch`, `mpc.gen`, ...) parser prepoznaje i preskače, tako da datoteka ostaje ispravan MATPOWER slučaj koji se može učitati i u MATLAB/Octave. V_m je magnituda napona faze u p.u., a V_a ugao u stepenima.

4 Arhitektura i implementacija

4.1 Struktura plugina

Plugin je izgrađen korištenjem `natID` SDK-a i prati standardnu strukturu `dTwin` dataka: `WindowPlugin` → `TabView` → `ViewConv`. Klasa `Plugin` implementira sučelje `sc::IPlugin` i izvozi se iz DLL-a funkcijom `getPluginInterface()`. Prozor sadrži pogled `ViewConv` sa svim kontrolama (izbor ulazne/izlazne datoteke, opcije konverzije, traka napretka i status).

4.2 Parsiranje ulaznog fajla

Parsiranje je realizovano konačnim automatom (*state machine*) sa stanjima `Searching`, `SkippingBlock`, `InBus3Ph` i `Done`. Automat traži trofazni blok, a ostale MATPOWER blokove preskače brojanjem dubine uglastih zagrada. Ključna funkcija za izdvajanje brojeva iz reda koristi `std::strtod` umjesto podjele stringa, čime se izbjegava pogrešno tumačenje višestrukih razmaka (kolonsko poravnanje) kao praznih polja:

```
static bool extractNumericRow(const td::String& row, td::UINT4 nExpectedCols,
                             cnt::PushBackVector<double>& flatValues,
                             td::String& status)
{
    const char* p = row.c_str();
    double values[kMaxSrcCols];
    td::UINT4 count = 0;
    for (;;) {
        while (*p==' ' || *p==',' || *p=='\t' || *p=='\r' || *p=='\n') ++p;
```

```

        if (*p == 0) break;
        char* pEnd = nullptr;
        double v = std::strtod(p, &pEnd);
        if (pEnd == p) break;          // nije broj -> kraj reda
        if (count < nExpectedCols) values[count] = v;
        ++count; p = pEnd;
    }
    if (count == 0) return true;        // prazan red
    if (count != nExpectedCols) { status = "ERROR!..."; return false; }
    for (td::UINT4 i=0; i<nExpectedCols; ++i) flatValues.push_back(values[i]);
    return true;
}

```

Nakon učitavanja, oba formata se preslikavaju u kanonski raspored (busId, Vm_A, Va_A, Vm_B, Va_B, Vm_C, Va_C), pa ostatak konvertora ne mora znati koji je format bio na ulazu.

4.3 Konverzija u radnoj niti

Sama pretvorba (4) obavlja se u zasebnoj radnoj niti, po čvoru i po fazi. Podaci se čuvaju u gustim (*dense*) matricama `dense::DblMatrix`, budući da su svi elementi popunjeni i nema matričnih operacija koje bi opravdale rijetki (*sparse*) format.

```

for (td::UINT4 node = 1; node <= model.nodeCount; ++node)
    for (td::UINT4 phase = 0; phase < kNumPhases; ++phase) {
        double Vm      = bus(node, 2 + phase*2);
        double VaDeg    = bus(node, 3 + phase*2);
        double VaRad    = VaDeg * kPi / 180.0;
        outVr(node, phase+1) = Vm * std::cos(VaRad);    // e = Vm*cos
        outVi(node, phase+1) = Vm * std::sin(VaRad);    // f = Vm*sin
        processedSteps.fetch_add(1);                    // za prikaz napretka
    }

```

4.4 Višenitnost i prikaz napretka

Zahtjev za prikazom napretka u realnom vremenu ostvaren je pomoću dvije niti:

- **radna nit** – izvršava konverziju i *jedina* upisuje brojač `processedSteps`;
- **nit napretka** – *jedina* čita taj brojač i periodično (svakih oko 30 ms) računa i prijavljuje procenat završenosti.

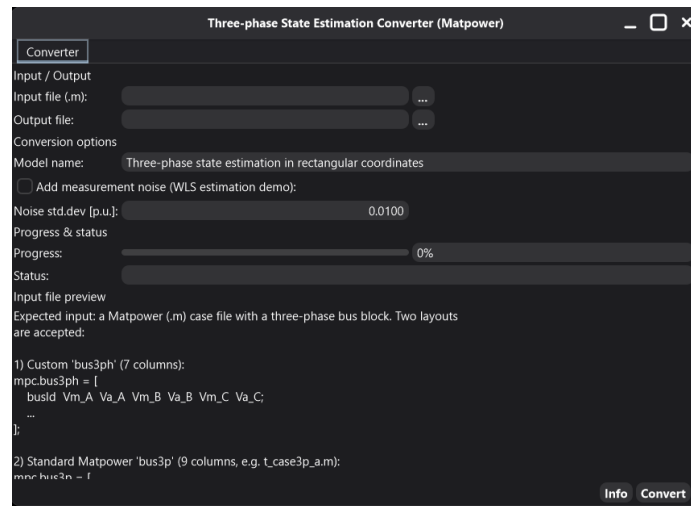
Budući da postoji tačno jedan pisac i jedan čitač za svaku dijeljenu veličinu, korišten je `std::atomic` bez potrebe za mutexom, čime se izbjegavaju utrke podataka (*data race*). Sam GUI se ne blokira: pogled pokreće ove dvije niti na pozadinskoj niti, a ažuriranja trake napretka i završetka se u glavnu (GUI) nit prosljeđuju preko `NatObject::asyncCall`.

4.5 Generisanje izlaznog modela

Rezultat konverzije se upisuje u `.dmodl` (digitalni model) i prateći `.vmodl` (vizuelni model / definicija grafa). Model se piše direktno u datoteku (klasa `FileSink`), a ne kroz trajnu memorijsku arhivu plugina. Time se izbjegava greška u kojoj bi uzastopne konverzije u istoj sesiji nadovezivale više modela u jednu datoteku (dva Header/Model bloka), što dTwin ne može otvoriti.

4.6 Korisnički interfejs

Interfejs (slika 1) organizovan je u logičke sekcije: ulaz/izlaz, opcije konverzije (naziv modela, opcija dodavanja mjernog šuma i njegova standardna devijacija), traka napretka sa statusom, te pregled ulazne datoteke. Dugme **Convert** pokreće konverziju, a **Info** učitava sadržaj ulazne datoteke.



Slika 1: Korisnički interfejs konvertora (ViewConv).

5 WLS estimacija u dTwinu

Konvertor *postavlja* problem estimacije, a *rješava* ga interni WLS solver alata dTwin (dugme **Solve**). Generisani `.dmodl` model sadrži blok **Vars** (promjenljive stanja e_i , f_i po fazi svakog čvora), blok **Params** (mjerenja i težine) i blok **WLSEs** (jednačine težinskih najmanjih kvadrata). Indeksiranje stanja je $i = (\text{čvor} - 1) \cdot 3 + \text{faza}$, pa za mrežu od N čvorova postoji $3N$ parova (e_i, f_i) .

5.1 Dodavanje mjernog šuma

Budući da su ulazne vrijednosti idealne (rezultat egzaktnog proračuna), bez šuma estimator nema šta da filtrira. Da bi se demonstrirala *stvarna* estimacija, opcionalno se

mjerenja generišu kao *tačna vrijednost* + *Gausov šum*, čime se oponaša ponašanje realnog mjernog uređaja:

$$z_{r,i} = e_{T,i} + \mathcal{N}(0, \sigma), \quad z_{i,i} = f_{T,i} + \mathcal{N}(0, \sigma). \quad (5)$$

Da bi estimator mogao filtrirati šum, svakoj se komponenti pridružuje *više nezavisnih* zašumljenih mjerenja (redundancija), pa je rješenje usrednjeni najbolji pogodak, a ne kopija jednog mjerenja.

5.2 Statistika greške

U bloku **PostProc** se za svako stanje akumulira apsolutna greška $|e_i - e_{T,i}|$ i $|f_i - f_{T,i}|$, te se računaju izlazni pokazatelji **minErr**, **maxErr**, **avgErr** i **devErr**. Kako se šum osvježava u svakom prolazu petlje **Repeats**, statistika greške efektivno predstavlja mali Monte-Carlo uzorak.

6 Rezultati i analiza

6.1 Test slučajeve

Korišteno je više test-slučajeva različite veličine i stepena nebalansiranosti (tabela 1).

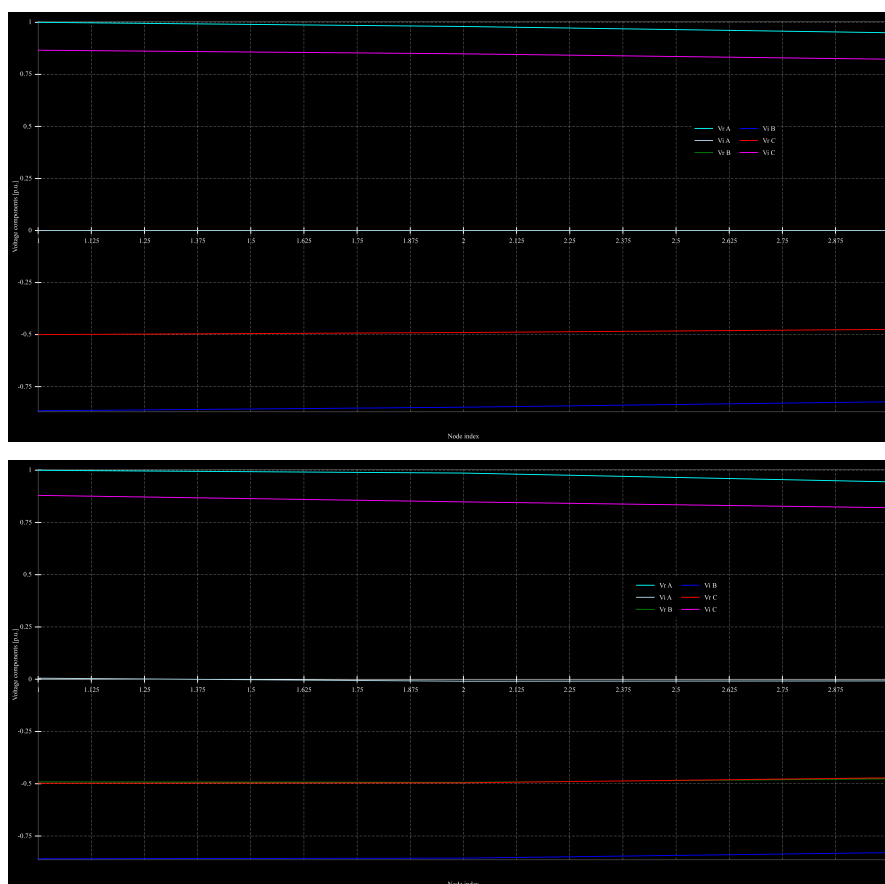
Tablica 1: Korišteni test-slučajeve.

Datoteka	Čvorova	Format	Karakteristika
case3_mini_3ph.m	3	bus3ph	balansiran, opadajuća magnituda
t_case3p_a.m	4	bus3p	standardni MATPOWER, balansiran
case9_3ph.m	9	bus3ph	blaga nebalansiranost
case14_3ph.m	14	bus3ph	vidljiva nebalansiranost

6.2 Provjera tačnosti konverzije

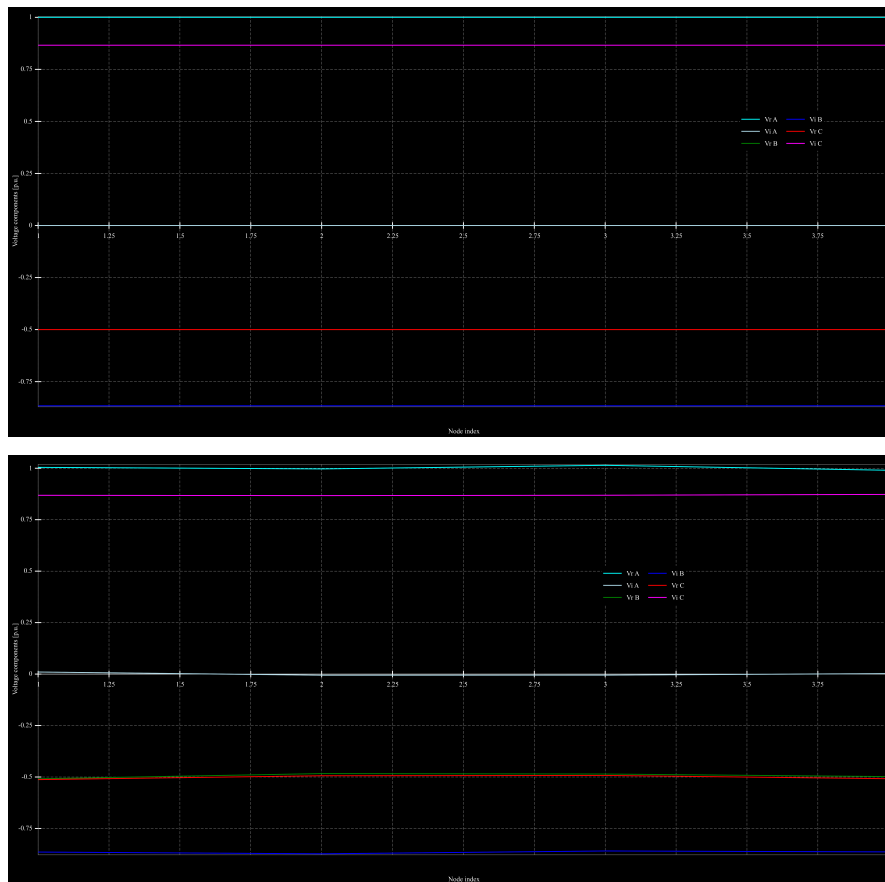
Za balansiran slučaj ($V_m = 1$, $V_a = 0^\circ / -120^\circ / +120^\circ$) konverzija daje egzaktne vrijednosti: faza A (1,0; 0,0), faza B (−0,5; −0,866), faza C (−0,5; +0,866), što je i ručno provjereno; maksimalno odstupanje bilo je reda 10^{-9} . Napominjemo da je $\cos(-120^\circ) = \cos(120^\circ) = -0,5$, pa se krive V_r^B i V_r^C potpuno preklapaju.

Slika 2 prikazuje slučaj case3 (balansiran): bez šuma su sve linije savršeno ravne (svi čvorovi imaju iste napone), dok se sa dodatim šumom javlja blago talasanje oko istih vrijednosti.



Slika 2: Naponske komponente za case3 (balansiran): gore bez šuma, dolje sa mjernim šumom.

Analogan rezultat za standardni MATPOWER format (**bus3p**, datoteka **t_case3p_a.m** sa 4 čvora) prikazan je na slici 3, čime je potvrđena i ispravnost parsiranja drugog ulaznog formata.

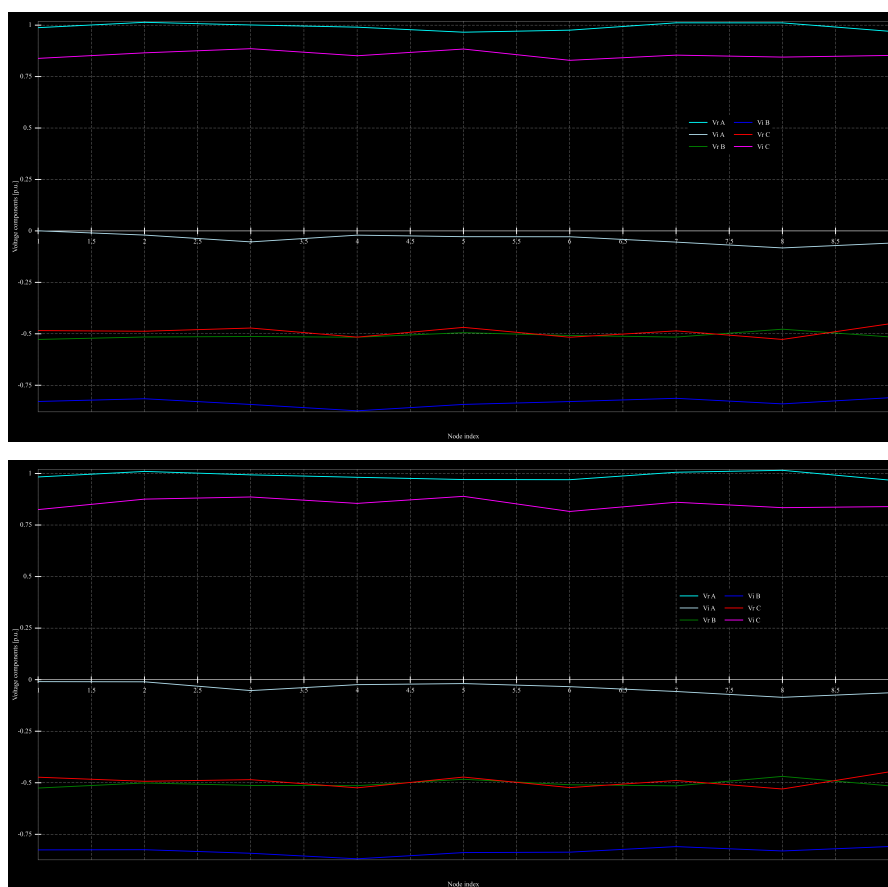


Slika 3: Standardni MATPOWER slučaj `t_case3p_a` (4 čvora): gore bez šuma, dolje sa mjernim šumom.

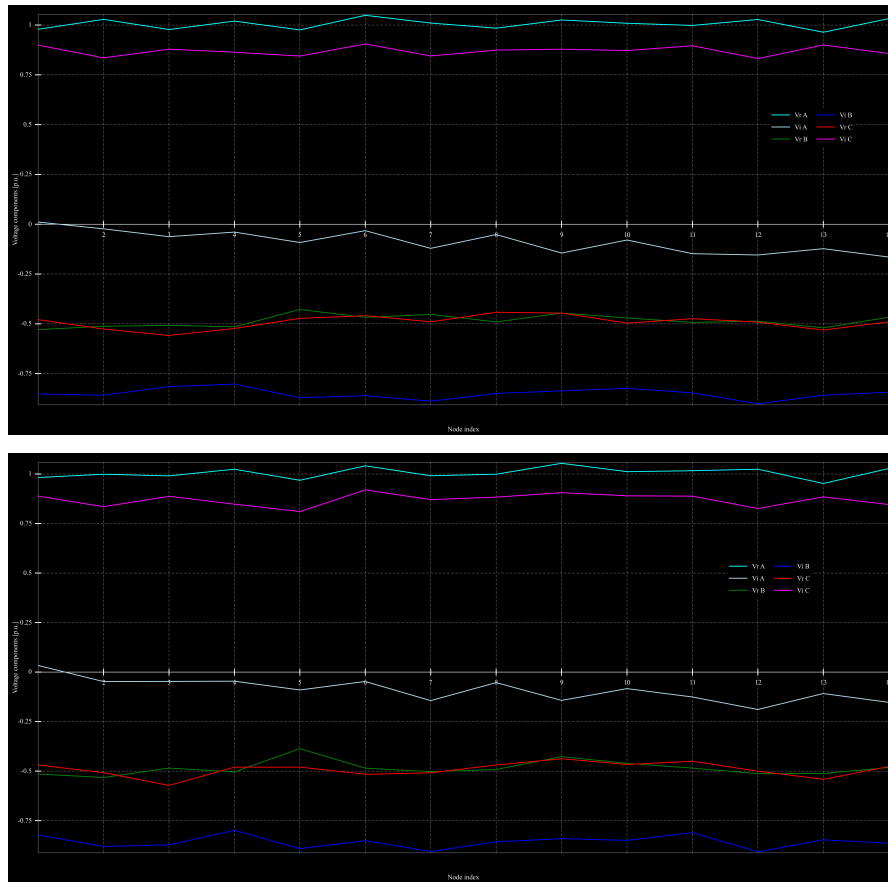
6.3 Nebalansirani slučajevi

Za razliku od balansiranih slučajeva (ravne linije), slučajevi `case9` i `case14` sadrže stvarnu nebalansiranost, pa su naponske krive prirodno talasaste i bez šuma – to je stvarni naponski profil mreže. Dodavanjem mjernog šuma krive postaju hrapavije, ali zadržavaju isti opšti oblik, jer estimator prepoznaje pravi profil ispod šuma.

Slika 4 prikazuje slučaj `case9` (9 čvorova, blaga nebalansiranost), a slika 5 slučaj `case14` (14 čvorova, izraženija nebalansiranost). Kod većih mreža je efekat nebalansiranosti dominantniji od šuma, pa se dva grafika (bez i sa šumom) više liče – šum tek blago „hrapavi“ već postojeće talasanje.



Slika 4: Naponske komponente po čvoru za case9 (9 čvorova): gore bez šuma, dolje sa mjernim šumom.



Slika 5: Naponske komponente po čvoru za case14 (14 čvorova): gore bez šuma, dolje sa mjernim šumom $\sigma = 0,03$.

6.4 Istorija rješavanja

Tabela rješenja u dTwinu (slika 6) prikazuje istoriju iteracija. Za N čvorova petlja **Repeats** pravi N prolaza (svaki sa svojim šumom), a svaki prolaz konvergira u dvije iteracije, pa se ispisuje po tri reda (Initial, IterBase, riješeno). Kolona **eps** prati konvergenciju.

	repeat#	outiter#	baselste...	totiter#	info	eps	e 1	f 1	e 2	f 2	e 3	f 3	e 4	f 4	e 5	f 5
1	0	1	0	0	Initial	1.00e+02	1.000000	0.000000	-0.500000	-0.966025	-0.500000	0.866025	1.000000	0.000000	-0.500000	-0.966025
2	0	1	1	1	IterBase	1.03e-02	0.991567	0.004705	-0.489675	-0.855887	-0.499409	0.859402	1.000721	0.004783	-0.501777	-0.871294
3	0	1	2	2	Initial	0.00e+00	0.991567	0.004705	-0.489675	-0.855887	-0.499409	0.859402	1.000721	0.004783	-0.501777	-0.871294
4	1	1	0	0	Initial	1.00e+02	0.991567	0.004705	-0.489675	-0.855887	-0.499409	0.859402	1.000721	0.004783	-0.501777	-0.871294
5	1	1	1	1	IterBase	1.97e-02	0.996706	0.001092	-0.487384	-0.866273	-0.497630	0.870921	0.995480	-0.007633	-0.499442	-0.867160
6	1	1	2	2	Initial	0.00e+00	0.996706	0.001092	-0.487384	-0.866273	-0.497630	0.870921	0.995480	-0.007633	-0.499442	-0.867160
7	2	1	0	0	Initial	1.00e+02	0.996706	0.001092	-0.487384	-0.866273	-0.497630	0.870921	0.995480	-0.007633	-0.499442	-0.867160
8	2	1	1	1	IterBase	2.62e-02	0.998716	0.016562	-0.506836	-0.831298	-0.500108	0.871835	1.005148	-0.001299	-0.492939	-0.871886
9	2	1	2	2	Initial	0.00e+00	0.998716	0.016562	-0.506836	-0.831298	-0.500108	0.871835	1.005148	-0.001299	-0.492939	-0.871886
10	3	1	0	0	Initial	1.00e+02	0.998716	0.016562	-0.506836	-0.831298	-0.500108	0.871835	1.005148	-0.001299	-0.492939	-0.871886
11	3	1	1	1	IterBase	2.48e-02	1.009429	0.002692	-0.507830	-0.876057	-0.487068	0.867919	1.004370	-0.001198	-0.493873	-0.869837
12	3	1	2	2	Initial	0.00e+00	1.009429	0.002692	-0.507830	-0.876057	-0.487068	0.867919	1.004370	-0.001198	-0.493873	-0.869837

Slika 6: Istorija rješavanja WLS problema (kolone e_i , f_i , eps).

6.5 Analiza greške estimacije

Ključni pokazatelj ispravnosti estimacije jeste odnos prosječne greške estimacije **avgErr** i jačine ubačenog šuma σ (tabela 2). U svim slučajevima je **avgErr** osjetno manji od σ , što dokazuje da WLS *filtrira* šum. Izlazni pokazatelji greške (**minErr**, **maxErr**, **avgErr**, **devErr**) koje model ispisi u rješenju prikazani su na slici 7.

No of errors:0		Cursor position:1,1		Log	Info	Res	▼
minErr	maxErr	avgErr	devErr				
0.000000	0.000000	0.000000	0.000000				
0.000000	0.000000	0.000000	0.000000				
0.002540	0.005367	0.003953	0.001444				
0.002540	0.005367	0.003953	0.001444				
0.002540	0.005367	0.003953	0.001444				
0.002540	0.018204	0.008027	0.006082				
0.002540	0.018204	0.008027	0.006082				
0.002540	0.018204	0.008027	0.006082				
0.000169	0.018204	0.005909	0.005861				
0.000169	0.018204	0.005909	0.005861				
0.000169	0.018204	0.005909	0.005861				
0.000169	0.018204	0.005047	0.005351				

Slika 7: Statistika greške estimacije: minimalna, maksimalna, prosječna i devijacija apsolutne greške $|e_i - e_{T,i}|$ kroz sve čvorove i prolaze.

Tablica 2: Prosječna greška estimacije u zavisnosti od jačine šuma.

σ (ubačeni šum)	avgErr (greška estimacije)	odnos
0,01	oko 0,0053	oko 53%
0,03	oko 0,0174	oko 58%

Rezultat se slaže s teorijom: usrednjavanje dva nezavisna mjerenja smanjuje standardnu devijaciju greške za faktor $\sqrt{2}$ (na oko $0,71\sigma$), a srednje apsolutno odstupanje Gausove raspodjele iznosi oko 0,8 te vrijednosti, što daje $\text{avgErr} \approx 0,57\sigma$ – upravo kako je i izmjereno.

7 Zaključak

U radu je realizovan konvertor trofazne estimacije stanja u pravougaonim koordinatama, isporučen kao DLL plugin za dTwin. Ispunjeni su svi postavljeni zahtjevi: konverzija $V, \theta \rightarrow V_r, V_i$ po fazi u zasebnoj radnoj niti, prikaz napretka u realnom vremenu na *thread-safe* način, te grafički interfejs sa plugin arhitekturom.

Uz osnovne zahtjeve, implementiran je i robustan parser koji podržava dva MAT-POWER formata, generisanje WLS modela sa pratećim grafom, te opcioni režim sa mjernim šumom u kojem estimator obavlja stvarnu estimaciju. Analiza je pokazala da je prosječna greška estimacije dosljedno manja od jačine ubačenog šuma, čime je potvrđeno ispravno filtriranje.

Kao mogući pravci proširenja izdvajaju se: uvođenje mrežne topologije (admitantne matrice) radi redundancije zasnovane na mjerenjima snaga i tokova, te podrška za veće mreže uz optimizaciju generisanja modela.